

LLM Destekli Kod Çevirisinde Sessiz Semantik Hatalar:

C'den Rust'a Çoklu-Model Deneyisel Bir Değerlendirme

Muhammet Baykara
Yazılım Mühendisliği
Fırat Üniversitesi
Elazığ, Türkiye
mbaykara@firat.edu.tr

İbrahim Halil Şahin
Yazılım Mühendisliği
Fırat Üniversitesi
Elazığ, Türkiye
230541112@firat.edu.tr

Emre Aşkın
Yazılım Mühendisliği
Fırat Üniversitesi
Elazığ, Türkiye
230541146@firat.edu.tr

Öz—Büyük dil modellerinin (LLM) ürettiği C→Rust çevirilerinin davranışsal eşdeğerliği yeterince incelenmemiştir. Bu çalışma, 57 C programı ve 233 test girdisinden oluşan özgün bir kıyaslama setinde, Claude Sonnet 5'in zero-shot çevirilerini diferansiyel test ile ölçer: ham yürütme doğruluğu (EA) yalnızca %70.18'dir (40/57); başarısızlıkların %94.1'i derleme aşamasında hiç yakalanmamış, %70.6'sı ise çalışma zamanında da hiçbir sinyal üretmeden yanlış çıktı veren sessiz semantik hatalardır. Hatalar dokuz tekrar eden C↔Rust semantik boşluğuna ayrılmıştır; ortak örüntü, modelin kaynağın özgül davranış sözleşmesi yerine hedef dilin idiyomatik varsayılanını seçmesidir. Kod uzunluğu tek başına açıklayıcı değildir. Deney altı yönde genişletilmiştir: ikinci bir modelle (Google Gemini, EA=%89.47) gerçek bir çoklu-model karşılaştırması, çok dosyalı/eşzamanlı C yapıları, Linux/LP64'te (Docker) 57 örneğin tamamında bir tekrar (54/57 PASS), ve sınırlandırılmış geri bildirimle bir iyileştirme deneyi. Model karşılaştırması tek bir modelin genel olarak “daha iyi” olmadığını gösterir: Gemini, Claude'un tekrarladığı beş kök-neden kategorisinden dördünü tamamen atlatırken, gerçek üretim kodu üçlüsünde (musl/Redis/cJSON) sıfır başarı göstermiştir; genel fark istatistiksel olarak anlamlıdır ama yön kategoriye göre değişir. Sessiz hata oranı da modele özgüdür: Claude'da %94.1 iken Gemini'de yalnızca %33.3'tür. Bulgular, LLM destekli kod çevirisinde asıl riskin sözdizimsel değil sessiz semantik hatalar olduğunu; hata geri bildiriminin doğruluğu %100'e taşıyabildiğini ama bunun zengin geri bildirimle dayanan bir üst sınır olduğunu, kısıtlı geri bildirimde %71.9-86.0'a düştüğünü göstermektedir. Tekrarlanabilirliği sınamak için yapılan bağımsız bir ikinci Claude denemesi (k=2), EA'yı %96.49'a taşımış ve k=1 ile sıfır ortak hata üretmiştir — tek-örneklemli EA'ların yüksek varyanslı olabileceğini doğrular göstermektedir; katman-eşleştirilmiş bir karşılaştırma (Claude Haiku, %63.16) ise Sonnet-Flash farkının yalnızca model katmanından kaynaklanmadığını doğrulamıştır.

Anahtar Kelimeler—kod çevirisi, büyük dil modelleri, Rust, diferansiyel test, semantik hata, çoklu-model karşılaştırma

I. GİRİŞ

A. Alan ve Önem

Kurumların onlarca yıllık C tabanlı yazılım envanteri, güvenlik ve bakım maliyeti açısından artan bir yükür; bellek güvenliği ihlallerinin (buffer overflow, use-after-free, veri yarışı) çoğu bellek-güvensiz dillerden kaynaklandığından, Rust gibi bellek-güvenli dillere geçiş kritik bir mühendislik hedefidir. Manuel geçiş maliyeti yüksek olduğundan, LLM tabanlı otomatik kod çevirisi hızla ilgi görmektedir; büyük teknoloji şirketleri ve kamu kurumlarının bu yöndeki açık politika duyuruları [1] bu ilgiyi somut bir mühendislik ihtiyacına dönüştürürken,

çevirinin doğruluğu yeterince doğrulanmadan kabul edilmesi riskini de beraberinde getirir.

B. Literatürdeki Boşluk

Bir LLM çevirisinin “başarılı” sayılması için hedef dilde derlenmesi yeterli değildir; kaynakla davranışsal olarak eşdeğer olmalıdır — sözdizimsel düzeyde doğru ama anlam düzeyinde sapan bir çeviri, derleyiciden hiç uyarı almadan üretime sızabilir. Bu çalışmanın çıkış noktası: LLM kod çevirisinde asıl tehlike sözdizimsel değil, sessiz semantik hatalardır; bu tehlikenin modelden modele nasıl değiştiği literatürde yeterince ölçülmemiştir.

C. Amaç ve Katkılar

Bu çalışma, büyük dil modellerinin ürettiği C→Rust çevirilerinin davranışsal doğruluğunu deneyisel olarak incelemektedir. Zero-shot çevirilerin yürütme doğruluğu (EA) ölçülmüş ve başarısızlıkların ne kadarının derleyiciden hiçbir uyarı almadan sessizce gerçekleştiği belirlenmiştir; başarısızlıkların kök nedenleri C↔Rust arasında tekrar eden semantik boşluklar olarak sınıflandırılmıştır. Ayrıca kod uzunluğunun çeviri başarısını gerçekten öngörüp öngörmediği, derleme yapılandırmasının (debug/release) ölçülen doğruluğu nasıl etkilediği ve hata geri bildirimine dayalı bir iyileştirme döngüsünün doğruluğu ne ölçüde artırdığı (ve bu artışın geri bildirimin zenginliğine ne kadar bağlı olduğu) sınanmıştır. Son olarak, bulguların tek bir modele mi özgü olduğu, yoksa ikinci bağımsız bir modelde (Google Gemini) de aynı örüntünün ve aynı kök nedenlerin gözlenip gözlenmediği araştırılmıştır.

Bu amaçla 57 C programı ve 233 test girdisinden oluşan özgün, tekrarlanabilir bir kıyaslama seti (“C→Rust Legacy Çeviri Benchmark'ı”) oluşturulmuş; her programın Claude ve Gemini çevirisi kaynağa karşı diferansiyel test ile doğrulanmıştır. Bu süreçte, bilinen C↔Rust semantik boşluklarını (tamsayı taşıması, string modeli, işaretlilik, biçimlendirme, global durum) hedefleyen özgün bir veri seti oluşturulmuş, derleme modunun doğruluğu nasıl gizleyebileceği gösterilmiş ve dokuz kök nedene dayanan bir hata taksonomisi çıkarılmıştır. Bunun yanında üç farklı geri bildirim seviyesinde iyileştirme döngüsünün etkinliği ölçülmüş, 13 bağımsız gerçek dünya algoritması (Rosetta Code, OpenBSD/FreeBSD libc, musl/Redis/cJSON) üzerinde tekrarlanan bir ölçüm yapılmış ve 57 örneğin tamamı üzerinde gerçek, otomatik bir ikinci-model (Gemini) değerlendirmesiyle modele özgü ve modelden bağımsız hata kategorileri birbirinden ayrılmıştır.

II. İLGİLİ ÇALIŞMALAR

LLM tabanlı kod çevirisinin doğruluğunu yürütme yoluyla doğrulayan çalışmalar artmaktadır. Gandhi vd. [2], COBOL→Java çevirisinde execution-guided refinement ile %81.99 doğruluğa ulaşmıştır; bu çalışma benzer bir mantığı C→Rust'ta benimser. Eniser vd. [3], FLOURINE ile kod uzunluğu arttıkça çeviri hatası riskinin arttığını gözlemlemiştir — bu çalışmanın bulguları bu gözlemle örtüşmez (Bölüm IV-B). Yin vd. [4], farklı LLM'lerin hata kök nedenlerinin büyük ölçüde benzer olduğunu göstermiştir; çoklu-model bulgularımız (Bölüm IV-F) bunu hem doğrular hem nüanslandırır: yalnızca kök neden D kategorisi modelden bağımsızken, A, B, C, E, F, G, H, I kategorilerinin her biri en az bir modelde başarıyla atlatılmıştır.

Kural tabanlı C→Rust çevirisi (ör. C2Rust [5]) ve nöral kod çevirisi (ör. TransCoder [6]) farklı bir eksenle konumlanır: kural tabanlı araçlar davranışsal eşdeğerliği sözdizimsel kurallarla (genellikle yoğun *unsafe* pahasına) garanti etmeye çalışırken, nöral/LLM yaklaşımlar deyimisel ama doğrulanmamış çeviriler üretir. Rust'ın tip sistemi ve ödünç denetleyicisi bellek güvenliğini derleme zamanında garanti eder, ama yalnızca kod derlenirse — davranışsal eşdeğerliği garanti etmez; bu çalışma tam olarak bu boşluğu, “derlenen ama kaynaktan farklı davranan” kodu, ölçmeyi hedefler.

TABLO I. VERİ SETİNİN KAYNAK/ROL KATEGORİLERİNE GÖRE ÖZETİ.

Kategori	Örnekler	Sayı	LoC	Kaynak
Temel algoritmalar	s01-s24	24	10-88	Özgün
Uzun özgün	s25-s29	5	69-141	Özgün
Rosetta Code	s30-s36	7	27-71	Bağımsız*
BSD libc	s37-s39	3	143-154	Bağımsız**
Hedeflenmemiş boşluk	s40-s45	6	21-43	Özgün
musl/Redis/cJSON	s46-s48	3	262-522	Bağımsız***
Kök-neden 2. örnek	s49-s53	5	27-33	Özgün
Çok dosyalı	s54,s55,s57	3	71-112	Özgün (çok dosya)
Karmaşık makro	s56	1	97	Özgün

a. * Rosetta Code; ** OpenBSD/FreeBSD libc; *** musl/Redis/cJSON — üçü de tarafımızca yazılmamış bağımsız/üretim kodu, bkz. Bölüm V-B.

B. Çeviri Süreci

Ana çeviri modeli Anthropic Claude Sonnet 5'tir (model kimliği: claude-sonnet-5); ikinci, bağımsız model olarak Google Gemini API'si [12] (gemini-flash-latest, gerçek API çağrısıyla, temperature=0.2, top_p=1.0) kullanılmıştır. Gemini kimliği sabit bir sürüm değil rotasyonlu bir takma addır (“-latest”); hangi anlık görüntüye (modelVersion) çözümlendiği o dönemde loglanmamıştır — tekrarlanabilirlik açısından kabul edilmesi gereken bir eksiklik (Bölüm V-B), ölçümün zamansal bağlamı yalnızca erişim tarihleriyle (2026-07-22 - 2026-07-25) yeniden kurulabilir. İstem mühendisliği en yalın haliyle uygulanmıştır: modele yalnızca “bu C programını Rust'a çevir” istemi verilmiş, model derleme/test aracına erişememiştir (gerçek zero-shot). Değerlendirme betiği model-bağımsız tasarlanmıştır. Claude çevirileri ise etkileşimli bir asistan oturumunda (Claude Code CLI) üretilmiş; örnekleme parametreleri ayrı arşivlenmemiştir;

Bu çalışma literatürden üç noktada ayrışır: (a) veri seti özgün ve tümüyle tekrarlanabilir; (b) derleme yapılandırmasının doğruluk üzerindeki gizli etkisi gösterilmiştir; (c) iki bağımsız model üzerinde gerçek, otomatik ölçümle kök neden düzeyinde bir model karşılaştırması sunulmuştur.

III. YÖNTEM

A. Veri Seti

Kıyaslama seti 57 C programından oluşur (10-522 satır), dokuz aşamada genişletilmiştir (Tablo I): 24 temel algoritma; kod uzunluğu etkisini test etmek için 5 daha uzun program (s25-s29); bağımsızlığı sınamak için Rosetta Code'dan [7] 7 algoritma (s30-s36), OpenBSD/FreeBSD libc'sinden [8] 3 üretim fonksiyonu (s37-s39) ve musl [9]/Redis [10]/cJSON [11]'dan 3 uzun üretim kodu (s46-s48); hedeflenmemiş boşlukları kapsayan 6 özgün program (s40-s45); beş kök-neden kategorisi (C, E, F, G, H) için sonradan eklenen bağımsız ikinci örnekler (s49-s53); ve çok dosyalı/thread eşzamanlılığını (s54, s55, s57) ve karmaşık makro kullanımını (s56) test eden iki ek grup. Çekirdek algoritma her durumda kaynaktan değiştirilmeden alınmış, yalnızca main() stdin/stdout sözleşmesine uyacak şekilde yeniden yazılmıştır; tam katalog açık veri deposunda sunulmuştur.

bu, tekrarlanabilirliği Gemini tarafına göre sınırlayan bir asimetridir (Bölüm V-B). Her iki model için de her örnek yalnızca bir kez (k=1) çevrilmiştir; bu tek-örneklemli ölçümün model farkını mı örnekleme gürültüsünü mü yansıttığı bilinmemektedir (Bölüm V-B). Gemini ölçümü, ücretsiz katman günlük kota sınırı nedeniyle 2026-07-22 - 2026-07-25 arasında kademeli tamamlanmıştır; çağrı kaydı (istem, zaman damgası, örnekleme parametreleri; modelVersion alanı hariç) 57 örneğin 38'i için açık veri deposunda arşivlenmiştir — manifest günlüğü sürecin ortasında etkinleştirildiğinden ilk 19 örneğin (s01-s19) çağrı kaydı yoktur (Bölüm V-B).

C. Diferansiyel Test ve Hata Taksonomisi

Hedef dilde önceden yazılmış birim test bulunmadığından, doğruluk kaynak programla aynı girdide çalıştırılıp çıktı karşılaştırılarak (diferansiyel test) ölçülmüştür: C ve Rust ikilileri derlenir (Rust derlenmezse CE); her test girdisi için beklenen/alınan çıktı karşılaştırılır

(zaman aşımı → NT, çökme → RE, fark → FE, eşitse PASS; Tablo II); bir örnek yalnızca *tüm* girdileri geçerse PASS sayılır.

TABLO II. HATA TAKSONOMİSİ.

Kod	Tür	Tanım
CE	Derleme Hatası	Rust kodu rustc ile derlenemiyor.
RE	Çalışma Zamanı Hatası	Program panik veriyor / çöküyor.
NT	Sonlanmama	Program zaman aşımında (5 sn) sonlanmıyor.
FE	Fonksiyonel Hata	Derlenip çalışıyor ama çıktı farklı.

D. Derleme Yapılandırması ve İyileştirme Döngüsü

Rust'ta tamsayı taşması debug modunda (overflow-checks açık) panik üretirken release modunda (-O) sessizce sarar; bu farkı göstermek için aynı Round 1 çevirileri her iki modda test edilmiştir. Round 1'de başarısız olan her örnek için hata sinyali modele geri verilip düzeltilmesi istenmiştir (Round 2); aynı 17 başarısızlık üzerinde, bilginin kısıtlandığı iki ek seviyeyle de ölçüm yapılmıştır (Seviye B: yalnızca derleyici/panik metni veya başarısız girdi; Seviye C: yalnızca başarısız test sayısı). Üç seviyenin tamamında yalnızca *bir* düzeltme turu uygulanmıştır (single-shot, yakınsayana kadar tekrarlanan bir döngü değil) — bu, Seviye B ve C'de EA'nın %100'e ulaşmamasının (sırasıyla %86.0, %71.9) nedenlerinden biridir: bazı düzeltmeler ilk turda başarısız kalmış ve yeniden denenmemiştir.

TABLO III. ÜÇ KOŞULUN KARŞILAŞTIRMASI (CLAUDE).

Koşul	Mod	EA	EA%	CE	RE	FE	NT
R1 — ham çeviri	Debug	40/57	70.18	1	4	12	0
R1 — ham çeviri	Release	42/57	73.68	1	2	12	0
R2 — iyileştirilmiş	Debug	57/57	100.0	0	0	0	0

A. Genel Yürütme Doğruluğu ve Sessiz Başarısızlıklar

Ham çeviriler için EA yalnızca %70.18'dir (Şekil 1). Round 1'de başarısız olan 17 örneğin yalnızca biri (s19, global durum) derleme aşamasında (CE) yakalanmıştır; geri kalan 16'sı (%94.1), 4 RE + 12 FE, derleyiciden hiç uyarı almadan sessizce geçmiştir (Şekil 2). Hiçbir örnek 5 sn'lik eşikte sonlanmama (NT) göstermemiştir; bu kategori taksonomide tanımlı olmakla birlikte üç koşulun hiçbirinde gözlenmemiştir.

Program düzeyinde %70.18 olan doğruluk, test girdisi düzeyinde %86.1'dir (198/230 — CE olan örneğin girdileri çalıştırılmadığından payda 230'dır); bu, program-düzeyi metriğin katı ("tüm girdileri geçmelidir") tanımından beklenen bir farktır. Daha çarpıcı olan: 12 FE örneğinin 6'sı (s06, s13, s15, s20, s27, s38) yalnızca *tek* bir sınır girdisinde sapmış, kalan girdilerin tamamında doğru sonuç üretmiştir; sessiz semantik hataların rastgele bir girdiyle yakalanma olasılığının düşük olduğunu, hatanın yalnızca sessiz değil seyrek de olduğunu gösterir.

E. Metrik ve Ortam

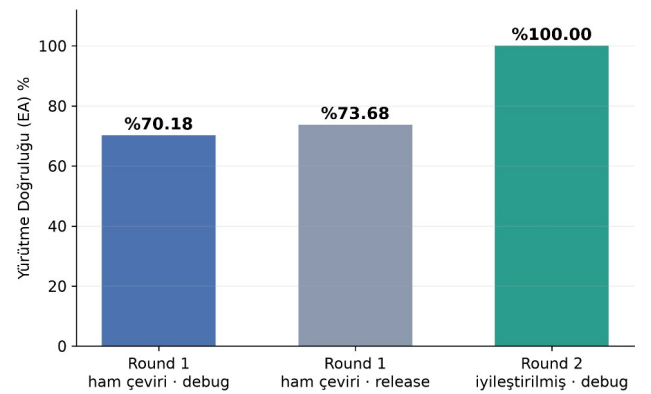
Program-düzeyi Yürütme Doğruluğu (EA) = tüm testleri geçen örnek sayısı / toplam örnek sayısı — ana metriktir; test-girdisi bazında geçme oranı ikincil bir metriktir. Ortam: Windows 11, gcc 16.1.0 (MSYS2/UCRT64, -O2), rustc/cargo 1.97.1; derleme rustc <dosya>.rs -o <çıktı> ile, Cargo veya --edition bayrağı olmadan yapılmıştır (varsayılan edition, 2015 — bazı lint'lerin şiddeti edition'a göre değişir; ancak Kategori E'deki E0133 derleme hatası tüm edition'larda sert hatadır). Ayrıca gerçek bir Linux/LP64 ortamında (Docker, ubuntu:24.04) veri setinin tamamı (57 örnek) üzerinde bir tekrar yapılmıştır (Bölüm IV-F). Zaman aşımı eşiği 5 sn'dir; yalnızca LLM çağrısı bire bir tekrarlanabilir değildir (Bölüm V-B). Tekrarlanabilirlik için gereken depo/erişim bilgileri Bölüm III-F'de sunulmuştur.

F. Veri, Kod ve Lisans Erişilebilirliği

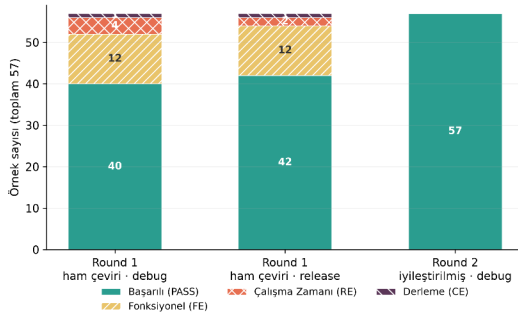
Tüm değerlendirme betikleri, veri seti, model çağrı kayıtları, istatistik ve platform-karşılaştırma raporları ile ham sonuçlar açık bir depoda erişime sunulmuştur. Depo bağlantısı ve kalıcı arşiv (Zenodo/figshare) DOI'si henüz atanmamıştır; bu bilgiler kabul sonrası yayın aşamasında paylaşılacaktır. Bağımsız kaynaklardan alınan programlar ilgili açık lisanslar altında yeniden kullanılmıştır: Rosetta Code örnekleri GFDL 1.2/CC-BY-SA; OpenBSD/FreeBSD libc ve Redis SDS kodu BSD-3-Clause; musl ve cJSON kodu MIT lisansı altındadır. Her kaynak dosyanın başlığında lisans bilgisi ve yapılan değişiklikler belgelenmiştir.

IV. BULGULAR

Deney üç koşulda çalıştırılmıştır (Tablo III): Round 1 debug, Round 1 release, Round 2 (iyileştirilmiş) debug.



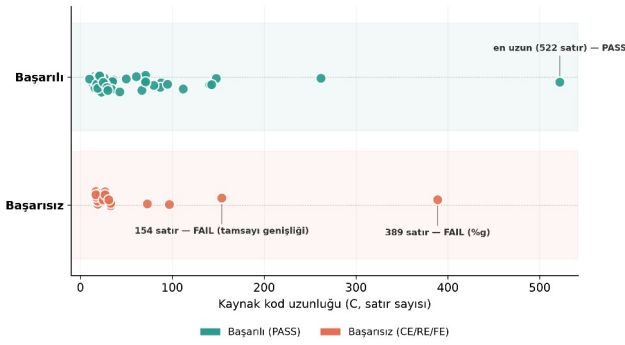
Şekil 1. Üç koşulda yürütme doğruluğu (sırasıyla %70.18, %73.68 ve %100).



Şekil 2. Koşullara göre sonuç dağılımı.

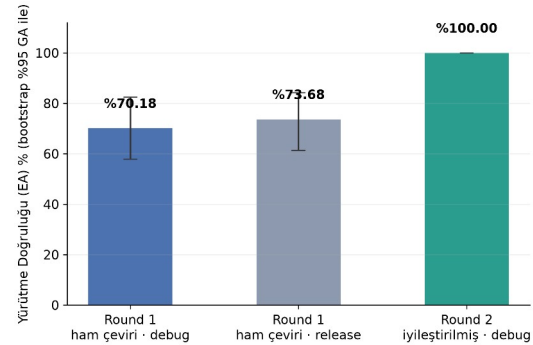
B. Kod Uzunluğu ile Başarı İlişkisi

Şekil 3, 57 örneğin LoC'sini başarı durumuna göre gösterir: en uzun program (522 satır, s47) geçerken çok daha kısa s38 (154 satır) ve daha kısa s48 (389 satır) başarısız olmuştur — başarısızlık uzunluktan çok hedeflenen semantik boşluğa bağlıdır. PASS (n=40) ve FAIL (n=17) gruplarının LoC dağılımı Mann-Whitney U testiyle karşılaştırılmış, fark anlamlı bulunmamıştır ($U=287.0$, $p=0.359$); veri seti genişletme aşamalarında (n=36→57 arası ara kontrollerde) p-değeri 0.076-0.359 arası dalgalanmıştır; küçük örnekleme p-değerinin istikrarsızlığının veriye dayalı bir göstergesi.



Şekil 3. Kod uzunluğu (LoC) ile başarı durumu ilişkisi (n=57).

İlişkinin etki büyüklüğü rank-biserial korelasyon ile $r=0.156$ 'dır (küçük etki). Post-hoc güç yerine (bu p-değerinin tekdüze bir dönüşümüdür ve bağımsız bilgi taşımaz [13]) duyarlılık analizi uygulanmıştır: $n(\text{FAIL})=17$, $n(\text{PASS})=40$, $\alpha=0.05$ ile %80 güçte saptanabilecek en küçük etki büyüklüğü, $\alpha=0.05$ 'te iki yönlü kritik U değerinden rank-biserial formülüyle ($r = 1 - 2U / (n1 \times n2)$) hesaplandığında $|r| \approx 0.46$ 'dır. Gözlemlenen $r=0.156$ bu eşğin belirgin altındadır — veri seti bu büyüklükte küçük-orta etkileri saptayacak güce sahip değildir; “anlamlı fark yoktur” sonucu düşük güçle tutarlı bir gözlem olarak okunmalıdır (Bölüm V-B). EA için de bootstrap %95 güven aralığı hesaplanmıştır (Şekil 4): Round 1 debug %70.18 [%57.89, %82.46]; geniş aralık, n=57'nin tek nokta tahminine duyulan güveni sınırladığını gösterir.



Şekil 4. Üç koşulda EA için bootstrap %95 güven aralığı.

Kod uzunluğunun tek başına açıklayıcı olmaması, başka hangi kod özelliklerinin ayırt edici olabileceği sorusunu gündeme getirir. Tablo IV, PASS/FAIL gruplarının işaretçi, dinamik bellek ayırma (malloc/calloc) ve string-fonksiyonu kullanım oranlarını gösterir (ölçüm tanımları regex tabanlı ve tekrarlanabilir; ayrıntılar açık veri deposunda belgelenmiştir).

TABLO IV. PASS/FAIL GRUPLARINA GÖRE BETİMSSEL KOD ÖZELLİKLERİ (n=57).

Özellik	PASS (n=40)	FAIL (n=17)
Ortalama LoC	67.0	59.8
Medyan LoC	34.5	27.0
İşaretçi kullanımı	%57.5	%41.2
malloc/calloc	%22.5	%0.0
String fonksiyonu	%37.5	%47.1

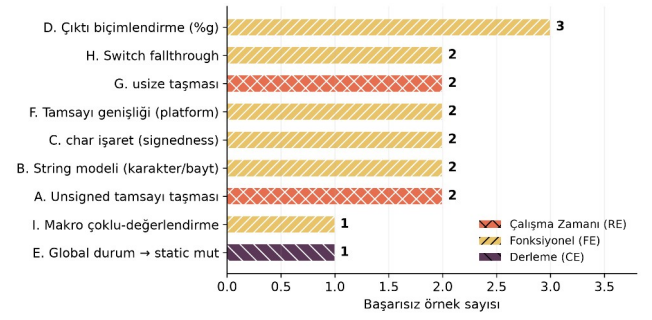
FAIL grubunun string-fonksiyonu kullanımı (%47.1) PASS grubundan (%37.5) yüksek, malloc/calloc kullanımı ise sıfırdır — bu, Bölüm IV-C'deki kök neden analiziyle tutarlıdır: başarısızlıkların çoğu string/karakter/sayı semantiğiyle ilgilidir; dinamik veri yapısı içeren programlar (bağlı liste, BST, hash tablosu) bu veri setinde istisnasız PASS olmuştur. İşaretçi kullanımı ile PASS/FAIL arasındaki ilişki Fisher'in kesin testiyle de sınanmış, anlamlı bulunmamıştır (tablo=[23,7],[17,10], odds oranı=1.93, %95 GA=[0.61, 6.11], $p=0.385$); güven aralığının 1.0'i içermesi ve genişliği, küçük örneklemin doğal bir sonucudur. Bu veri setinde küçük örneklem büyüklüğü nedeniyle aynı veriler üzerinde birden fazla çıkarımsal test (Mann-Whitney, Fisher, McNemar) uygulanmıştır; bootstrap güven aralığı bir hipotez testi olmadığından bu sayıma dahil değildir. Bunlar keşif amaçlı (exploratory) okunmalı ve bir

çoklu-karşılaştırma düzeltmesiyle birlikte değerlendirilmelidir (ayrıntılı düzeltme kontrolü: Bölüm V-B).

C. Kök Neden Analizi

17 başarısızlık 9 kök nedene ayrıştırılmıştır (Şekil 5, Tablo V): 8 kategoride (A-D, F-I) Rust kodu sözdizimsel olarak geçerlidir; model, kaynağın bit-genişliği/işaretililik/sarma/kontrol-akışı sözleşmesini korumak yerine hedef dilin idiyomatik varsayılanını seçmiştir. Tek istisna kategori E'dir: aynı türden bir birebir aktarım Rust'ın tip güvenliği tarafından derleme aşamasında engellenmiştir — 17 başarısızlığın tek CE'si. Bu tanım (“derleyici uyarısı almadan geçme”) tek katmanlıdır: 4'ü aslında gürültülü, kolay fark edilen bir çalışma zamanı paniğidir (RE); gerçek anlamda sessiz olanlar yalnızca 12

FE'dir (%70.6): derlenir, çöküp panik vermeden çalışır, ama yanlış çıktı üretir. Daha doğru çerçeveleme iki katmanlıdır: “derleme aşamasında yakalanmamış” (%94.1) ve “çalışma zamanında da sinyal vermeden yanlış sonuç veren” (%70.6).



Şekil 5. Başarısızlıkların kök-neden dağılımı (Round 1, s49-s53'ün bağımsız 2. örnekleri dahil).

TABLO V. DOKUZ KÖK NEDEN VE TEMSİLİ ÖRNEKLER.

Kod	Kategori	Örnek	Boşluk (C sözleşmesi → LLM seçimi)
A	Unsigned taşma	s09,s14(RE)	Mod-2 ⁿ sarma → panik-edem varsayılan aritmetik
B	String modeli	s06,s13(FE)	Bayt-düzeyi C string → .chars() (Unicode)
C	char işaretliliği†	s20,s49(FE)	İşaretli char (128 ve üzeri negatif) → u8 (hep pozitif)
D	Çıktı biçimlendirme	s15,s27,s48(FE)	%g mantığı → Rust'ın varsayılan {} biçimi
E	Güvensiz global durum	s19(CE)	Değiştirilebilir global → static mut (unsafe)
F	Platform tamsayı genişliği†	s38,s51(FE)	Gerçek long genişliği (32-bit) → varsayılan i64
G	usize taşması	s40,s52(RE)	İşaretli int ile güvenli n-1 → usize altında taşma
H	Switch fallthrough	s43,s53(FE)	Kümülatif düşme → bağımsız match kolları
I	Makro çoklu-değerlendirme	s56(FE)	Metinsel ikame → fn (tek değerlendirme)

a. s09,s14,s20 vb. veri setinin özgün ilk turundandır; s49,s51,s52,s53 ilgili kategoriler için sonradan eklenen bağımsız 2. örneklerdir. † C ve F kategorilerinde “doğru” davranış referans platformun (Windows/LLP64) implementation-defined seçimine bağlıdır — bkz. Bölüm V-B. E kategorisinin ikinci örneği (s50) Round 1'de PASS olduğundan bir başarısızlık örneği olarak burada listelenmemiştir.

Doğrulama iki biçimde gerçekleşmiştir: (a) dört kategori (C, F, G, H) farklı bir kod deseni kullanan ikinci bir örnekle (s49, s51, s52, s53) yeniden sınanmış, kök neden her dördünde tekrarlanmıştır (Bölüm IV-F); (b) kategori D üç bağımsız kod tabanında (iki özgün örnek + cJSON [11]) tekrarlanmış, kategori A ise veri setinin kendi içinde iki bağımsız örnekte (s09, s14) tekrarlanmıştır. (b) maddesindeki üç bağımsız kod tabanı bulgusu, bu boşlukların veri setine özgü değil sistematik C↔Rust farkları olduğunu göstermektedir.

D. Derleme Modu ve İyileştirme Döngüsünün Etkisi

TABLO VI. ÜÇ GERİ BİLDİRİM SEVİYESİNİN KARŞILAŞTIRMASI, CLAUDE.

Seviye	İçerik	EA% (Claude)
A (Oracle)	Tam derleyici/panik metni + beklenen/alınan farkı	100.0
B (CI-benzeri)	Derleyici/panik metni tam; FE'de yalnızca girdi	86.0
C (Minimal)	Yalnızca başarısız test sayısı (CE hariç)	71.9

a. n=57; yalnızca Round 1'de başarısız olan 17 örnek yeniden çevrilmiş, EA tüm 57 program üzerinden yeniden hesaplanmıştır.

E. Gerçek Dünya Kodu ve Bellek Güvenliği

13 bağımsız gerçek dünya programının 11'i (%84.6) ilk seferde geçmiştir: Rosetta Code 7/7, BSD libc 2/3, musl/Redis/cJSON 2/3 (Claude). En uzun örnek (522 satır, s47) PASS olurken çok daha kısa s48 (389 satır) başarısız olması, uzunluk değil boşluğun belirleyici olduğunu burada da doğrular. Bellek güvenliği tarafında, Claude'un ürettiği 74 farklı çeviriden (57 Round 1 + Seviye A geri bildirimiyle yeniden çevrilen 17 Round 2 örneği; Seviye B ve C çevirileri bu sayıma dahil değildir) yalnızca 4'ü (%5.4)

unsafe içerir: s37_bsd_getopt, s44_fib_memo_static, s46_musl_qsort, s50_id_generator — dördü de Round 1'de PASS olduğundan Round 2'de yeniden çevrilmemiştir. Kategori E aynı model tarafından iki farklı biçimde çözülmüştür: s50_id_generator'da unsafe bloğu doğru eklenerek ilk seferde PASS olunurken, s19_global_counter Round 2'de unsafe kullanmadan (global sayaç yerine &mut i32 parametresi geçirilerek) düzeltilmiştir; bu, Kategori E'nin unsafe gerektirmeden de çözülebildiğinin somut kanıtıdır. Ham işaretçi aritmetiğine bağımlı heapsort (s39)

ve Redis'in SDS kütüphanesi (s47) bile tamamen güvenli, deyimisel Rust'a çevrilmiştir.

F. Çoklu-Model Karşılaştırması (Gemini)

Gemini ile 57 örneğin tamamı üzerinde gerçek, otomatik ölçüm yapılmıştır: EA=%89.47 (51/57; 4 CE, 2 FE, 0 RE), Claude'dan (%70.18) yüksektir; iki model *aynı* 57 program üzerinde ölçüldüğü için eşleştirilmiş McNemar testiyle sınıandığında fark anlamlıdır (yalnızca Claude'un başarısız olduğu 14 vs. yalnızca Gemini'nin başarısız olduğu 3 örnek; $p=0.013$). Ancak Tablo VII'deki kategori-bazlı kırılım bu genel farkın kategoriye özgü olduğunu ve yön değiştirebildiğini gösterir. Ayrıca Gemini'nin 6 başarısızlığının 4'ü (s26, s27, s46, s48) CE'dir, yani derleyici tarafından yakalanmıştır — Claude'da gözlenen “çoğu sessizdir” örüntüsü Gemini için aynı güçte geçerli değildir; sessiz hata oranı modele göre değişir.

Bölüm V-B'de belirtilen bir sınırlamayı — Claude Sonnet 5'in üst-düzey (frontier), gemini-flash-latest'in ise hafif/hızlı katman bir model olması nedeniyle karşılaştırmanın adil olmayabileceği eleştirisini — doğrudan sınamak için, aynı 57 örnek katman-eleştirilmiş bir üçüncü

modelle, Claude Haiku 4.5 ile de çevrilmiştir (taze, bu çalışmanın bulgularından habersiz izole bir oturumda; yöntem ve doğrulama Bölüm IV-G'dekiyle aynıdır). Sonuç: EA=%63.16 (36/57; 2 CE, 2 RE, 17 FE) — hem Sonnet'in (%70.18) hem de Gemini Flash'ın (%89.47) belirgin altında (Tablo VII, 4. sütun). Bu, Sonnet-Flash farkının yalnızca “üst-düzey vs. hafif katman” karşılaştırmasından kaynaklanmadığını gösterir: katman eşleştirildiğinde de Gemini Flash, Claude'un hafif modelinden çok daha yüksek doğruluk göstermektedir (aradaki fark %26.3 puan — Sonnet-Flash farkından, %19.3 puan, bile daha büyüktür). Ayrıca Haiku'nun 2 derleme hatası, Sonnet'in 17 başarısızlığının hiçbirinde görülmeyen tamamen yeni bir türdendir: Rust'ın ödünç denetleyicisini (borrow checker) ihlal eden sözdizimsel/mechanik hatalar (ör. E0716 “temporary value dropped while borrowed”, E0502 “cannot borrow as mutable because also borrowed as immutable”) — C↔Rust semantik boşluklarından değil, doğrudan Rust'ın kendi tip/ödünç sistemini doğru uygulayamamaktan kaynaklanır. Bu, daha küçük modellerin yalnızca dokuz semantik kategoride değil, Rust'a özgü mekanik kurallarda da zorlanabileceğini düşündürür.

TABLO VII. MODEL × KATEGORİ KIRILIMI.

Kategori (n)	Claude	Gemini	Haiku
Temel algoritmalar (24)	17/24	23/24	14/24
Uzun özgün (5)	4/5	3/5	2/5
Rosetta Code (7)	7/7	7/7	7/7
BSD libc (3)	2/3	3/3	0/3
Hedeflenmemiş boşluk (6)	4/6	6/6	5/6
musl/Redis/cJSON (3)	2/3	0/3	1/3
Kök-neden 2.örnek (5)	1/5	5/5	4/5
Çok dosyalı (3)	3/3	3/3	3/3
Karmaşık makro (1)	0/1	1/1	0/1

a. 57/57, üç model için de tam.

Üç örüntü dikkat çekicidir. Birincisi, “kök-neden 2. örnek” grubunda (s49-s53: Claude'un Round 1'de yalnızca birer örnekle temsil ettiği C, E, F, G, H kategorileri için bağımsız ikinci sınama) Claude bu beş örneğin dördünde aynı kök nedeni tekrarlamış, yalnızca birinde geçmiştir (1/5) — sistematik bir model-özgü kör nokta kanıtıdır. Gemini ise tamamında (5/5) farklı deyimisel tercihlerle (ör. wrapping_mul, AtomicI32) bu kategorileri sistemli biçimde atlatmıştır. İkincisi, tam tersi yönde: en uzun/karmaşık üç gerçek üretim kodu örneğinde (musl qsort, Redis SDS, cJSON, 262-522 satır) Claude 2/3 geçerken Gemini 0/3 geçmiştir. Üçüncüsü, Claude'un Round 1'deki tek makro-kaynaklı başarısızlığı (s56, kategori I) Gemini tarafından ilk seferde doğru çevrilmiştir (1/1): küçük ama ek bir kanıt. Bu üç bulgu birlikte, “bir model diğerinden genel olarak daha iyidir” iddiasının yanlış çerçeveleme olduğunu gösterir; Tablo VII'de betimsel olarak gözlenen kırılım, model ile kod kategorisi arasında bir etkileşime işaret etmektedir (istatistiksel olarak test edilmemiştir, bkz. Bölüm V-B); kısa sızıntı kategorilerinde Gemini, uzun/karmaşık üretim kodunda Claude daha dayanıklıdır.

Gemini'nin Claude'da hiç görülmeyen üç başarısızlığı vardır: ikisi (s46_musl_qsort, s47_redis_sds) yukarıdaki

üretim kodu üçlüsüne aittir — s46'da geçersiz bir işaretçi söz dizimi (*mut u8::null()), s47'de yanlış büyüme mantığı (LEN=0) üretilmiştir (Claude aynı ikisini sırasıyla sınırlı unsafe ve hiç unsafe kullanmadan doğru çevirmiştir, Bölüm IV-E); üçüncüsü (s26_rpn_calculator) bir ödünç denetleyicisi hatasıdır (E0499, iki FnMut kapanışının aynı yığın işaretçisini yakalaması), Claude farklı bir kontrol akışıyla bu hataya hiç maruz kalmamıştır. İkisi derleme hatası (s46, s26), biri fonksiyonel hatadır (s47); bu karışım, tek bir kök-neden taksonomisinin tüm modellere aynı biçimde uygulanamayabileceğini gösterir. Her iki modelin de başarısız olduğu üç örnek de vardır: s15 aynı kök nedenden (kategori D); s27_csv_stats ve s48_cjson_number'da ise farklı türde — Claude'da derlenen ama yanlış sonuç veren kod (FE), Gemini'de geçersiz söz dizimi (CE: s27'de {; +=03} biçim belirteci, s48'de işaretçi bildirimi hatası). Kategori D'nin üç bağımsız kod tabanının (s15, s27, s48) üçünde de başarısızlık, bu boşluğun modele özgü olmadığını doğrular; çok dosyalı/eşzamanlı yapıda (s54, s55, s57) ise her iki model de ilk seferde başarılıdır (n=3, model farkı gözlenmemiştir).

Son olarak, deney gerçek bir Linux/LP64 ortamında (Docker) veri setinin tamamı (57 örnek) üzerinde bir tekrar

ile de genişletilmiştir. Bu, en önemli ek bulguyu vermiştir: Windows'ta ölçülen %100 Round-2 doğruluğu Linux'ta %94.74'e (54/57) düşmüştür; aynı yön Round 1'de de gözlenir (debug %70.18→%71.93, release %73.68→%75.44 — Linux'ta hafifçe daha yüksek EA, aşağıda açıklanan üç örneğin ikisinin bu koşullarda platform lehine dönmelerinden kaynaklanır). Veri setine sonradan eklenen s56 ve s57, üç koşulun hiçbirinde platforma bağlı bir fark göstermemiştir — bu, aşağıdaki üç örneğin platforma duyarlılığının veri setine özgü bir tesadüf değil, hedeflenen belirli semantik boşluklara (tamsayı genişliği, stdio metin modu) özgü olduğunu güçlendirir. Üç örnekte farklı sonuç gözlenmiştir: s38 ve s51'in Round 2 düzeltmeleri 32-bit `long` varsayımına dayandığından Linux'un 64-bit `long`'unda geçersiz kalmıştır (bir platformda doğrulanmış bir düzeltme başka platformda geçersiz olabilir); s47 `redis_sds`'teki neden ise Rust çevirisiyle ilgili değildir — bir test girdisi (05.txt) CRLF içerir ve Windows'un C çalışma zamanı bunu `\r\n`→`\n`'e çevirirken Linux/glibc çevirmez, yani C referans programının kendisi (Rust çevirisi değil) iki platformda farklı sayıda komut tüketir. Yani hem “düzeltilmiş bir çeviri evrensel doğrudur” varsayımı hem de referans C programının platformlar arası tam taşınabilirliği yanlış olabilir.

G. Tekrarlanabilirlik: *pass@k* Ölçümü

Bölüm V-B'de $k=1$ ölçümünün en önemli metodolojik sınırlama olduğu belirtilmişti: her örnek yalnızca bir kez çevrildiğinden, raporlanan EA'nın gerçek model performansını mı yoksa örnekleme gürültüsünü mü yansıttığı bilinmiyordu. Bunu doğrudan sınamak için, aynı 57 C programı, bu çalışmanın bulgularından ve önceki çevirilerden tamamen habersiz, izole bir Claude Sonnet 5 oturumuyla ikinci kez ($k=2$) çevrilmiştir; izolasyon, oturumun yalnızca `samples_c/` altındaki kaynak dosyaları okuduğu ve yalnızca kendi çıktı dizinine yazdığı, başka hiçbir depo dosyasına erişmediği tam işlem kaydı (transcript) denetlenerek doğrulanmıştır. Sonuç çarpıcıdır: EA=%96.49 (55/57) — $k=1$ 'in (%70.18) 26.3 puan üzerinde. Daha da dikkat çekici olan, iki koşul arasındaki örtüşmenin sıfır olmasıdır: $k=1$ 'in 17 başarısızlığının tamamı $k=2$ 'de düzelmiş, ama $k=2$ 'de daha önce hiç görülmemiş 2 yeni başarısızlık (s47 `redis_sds`, s54 `stack_module`) ortaya çıkmıştır. Bu ölçüm, $k=1$ ile aynı arayüz/örnekleme koşulları altında yapılmamıştır (Bölüm III-B'de belirtildiği gibi, ne orijinal Claude Code CLI oturumunun ne de bu izole ajan oturumunun örnekleme parametreleri loglanmıştır); bu nedenle gözlenen farkın ne kadarının gerçek model stokastisitesinden, ne kadarının iki farklı çalışma ortamından geldiği kesin olarak ayrıştırılamaz — ama bu belirsizliğin kendisi bile, tek bir $k=1$ ölçümünün ne kadar kırılgan bir temel olabileceğini doğrudan göstermektedir.

$k=2$ 'nin iki yeni başarısızlığı farklı iki nedenden kaynaklanır ve ikisi de öğreticidir. s47 `redis_sds`'te $k=2$ çevirisi, $k=1$ 'in kullandığı yüksek-seviye Rust dize/satır API'leri (`split whitespace`, `BufRead::lines()`) yerine C'nin `getchar()`'ını birebir taklit eden elle bayt-düzeyi bir ayrıştırıcı yazmayı tercih etmiş; bu ayrıştırıcı CRLF (CR+LF, iki baytlık) satır sonunu tek bayt sanıp yalnızca CR baytını tüketmiş, böylece test 05.txt'te (Bölüm IV-F'de tartışılan CRLF dosyasıyla aynı dosya) baştan sahte bir boş

komut satırı üretmiştir — C↔Rust boşluğu değil, saf bir uygulama tercihi kaynaklı yeni bir hata. s54 `stack_module`'deki neden ise bir model hatası değil bir veri seti tasarım boşluğudur: `STACK_CAPACITY` sabiti `stack.h`'de tanımlıdır, ama çok-dosyalı istem inşası (Bölüm III-A) yalnızca `manifest.json`'da listelenen `.c` dosyalarını (`stack.c`, `main.c`) modele verir — başlık dosyası hiçbir zaman gösterilmez. $k=1$ bu gizli sabiti doğru tahmin etmiş (100), $k=2$ farklı bir tahminde bulunmuş (128) ve kapasite-sınırı testi (04.txt) bu yüzden başarısız olmuştur. Aynı gizli-başlık durumu s55 `config_parser`'da da vardır (`config.h`, `MAX_KEY_LEN/MAX_VAL_LEN`), ama test seti bu sınırları hiç zorlamadığından iki koşulda da fark edilmemiştir — bu, veri setinin çok-dosyalı örnekleri için ayrıca belgelenmesi gereken bir kapsam boşluğudur (Bölüm V-B). Sonuç olarak bu tekrar, hem $k=1$ ölçümünün varyans endişesini somut veriyle doğrulamakta (aynı model, aynı istem, sıfır ortak hata) hem de gözlenen farkın bir kısmının saf model stokastisitesinden değil, kısmen istemin/veri setinin kendisindeki belirsizlikten (gizli başlık sabitleri) kaynaklandığını göstermektedir. Makalenin ana %70.18 rakamı bu nedenle tek bir örneklem olarak okunmalı; gerçek kapasiteyi tahmin etmek için $k \geq 3$ tekrarlı bir *pass@k* protokolü gerekir (Bölüm V-B).

V. TARTIŞMA

A. Bulguların Özeti ve Değerlendirmesi

Bu çalışmanın merkezi bulgusu, incelenen modellerin (Claude, Gemini) ürettiği C→Rust çevirilerinde asıl riskin sözdizimsel değil sessiz semantik hatalar olduğudur; Claude'da 17 başarısızlığın 16'sı derleyiciden hiç uyarı almadan geçmiştir. İkinci gözlem: derleme yapılandırması (debug/release) ve platform (Windows/Linux) gizli birer değişkendir — aynı kod farklı yapılandırmada farklı sonuç verebilir. Üçüncü, en yeni gözlem: iki bağımsız modelin karşılaştırması “model A, B'den genel olarak daha iyidir” biçiminde tek boyutlu bir sıralamanın yanıltıcı olduğunu gösterir (genel EA farkı anlamlı olsa da, $p=0.013$, Bölüm IV-F); Gemini kısa hedeflenmiş kategorilerde (5/5) Claude'dan (1/5) daha dayanıklı, uzun/karmaşık üretim kodunda (0/3) Claude'dan (2/3) daha kırılgandır; model seçimi kod tabanının niteliğine göre yapılmalıdır. Dördüncü gözlem: kod uzunluğu ile başarı arasında anlamlı bir ilişki bulunmamıştır ($p=0.359$); semantik karmaşıklık, ham uzunluktan daha belirleyicidir. Beşinci, en çarpıcı gözlem: $k=1$ ölçümünün tekrarlanabilirliği doğrudan sırandığında (Bölüm IV-G), bağımsız bir ikinci deneme EA'yı %70.18'den %96.49'a taşımış ve sıfır ortak hatayla tamamen farklı bir başarısızlık kümesi üretmiştir — tek-örneklemli EA rakamlarının (bu çalışmanın kendi başlık rakamı dahil) yüksek varyanslı bir dağılımdan tek bir çekiliş olarak okunması gerektiğini göstermektedir. Altıncı gözlem: katman-eşleştirilmiş karşılaştırma (Claude Haiku vs. Gemini Flash, Bölüm IV-F) Sonnet-Flash farkının yalnızca model katmanından kaynaklanmadığını doğrulamış, ayrıca daha küçük modellerin C↔Rust semantik boşluklarının ötesinde Rust'ın ödünç denetleyicisiyle de mekanik olarak zorlanabileceğini göstermiştir.

B. Sınırlamalar

İç geçerlilik: test girdileri elle seçilmiş, referans çıktılar C programı çalıştırılarak üretilmiştir; sınır durumu kapsamı ve kod kapsamı (code coverage) sistematik ölçülmemiştir; en uzun/karmaşık üç üretim örneği (s46, s47, s48) için ek

sınır-durum testleriyle girdi sayısı 5'ten 6-7'ye çıkarılmış olsa da (tümü PASS kaldı, bkz. veri deposu), 522 satırlık s47_redis_sds gibi örneklerde bu kapsam hâlâ küçüktür. İki model ayrıca farklı arayüzlerde koşturulmuştur (Claude etkileşimli CLI, Gemini doğrudan API); bu yalnızca bir tekrarlanabilirlik asimetrisi değil bir karıştırıcı değişkendir (confounder) — McNemar farkının ne kadarının modelden, ne kadarının koşum ortamından geldiği ayrıştırılamamıştır. Ayrıca karşılaştırılan iki model aynı kapasite katmanında değildir: Claude Sonnet 5 üst-düzye (frontier) bir model iken gemini-flash-latest Google'ın hafif/hızlı katman modelidir; bu nedenle gözlenen fark kısmen “Claude vs. Gemini” değil “üst-düzye vs. hafif katman” karşılaştırmasını yansıtır olabilir — bu doğrudan sınanmış (Bölüm IV-F): katman-eleştirilmiş Claude Haiku 4.5 (%63.16) Gemini Flash'ın (%89.47) yine de belirgin altında kalmıştır, yani Sonnet-Flash farkı yalnızca katman farkından ibaret değildir — ama Gemini Pro/Ultra ile simetrik bir karşılaştırma hâlâ eksiktir. C her koşulda -O2 ile derlenirken Rust debug/release ayrılmıştır (bilinçli tasarım kararı); C'nin farklı optimizasyon seviyelerinde tekrarı ayrı bir sağlamlık kontrolü olurdu. Her örnek yalnızca bir kez ($k=1$) çevrilmiştir; bu sınırlamanın somut etkisi Bölüm IV-G'de bağımsız bir $k=2$ ölçümüyle doğrudan gösterilmiştir (EA %70.18→%96.49, sıfır ortak hata) — gerçek kapasiteyi güvenilir tahmin etmek için yine de $k \geq 3$ tekrarlı bir protokol gerekir. Ayrıca Gemini çağrı kayıtlarının (istem, zaman damgası, örnekleme parametreleri) yalnızca 38/57'si (%67) açık veri deposunda arşivlenmiştir; ilk 19 örnek (s01-s19) manifest günlüğü etkinleştirilmeden önce çevrildiğinden bu örnekler için hiçbir çağrı kaydı yoktur — bu, Bölüm III-B'de belirtilen arşivleme kapsamını daraltan ek bir tekrarlanabilirlik eksikliğidir. İstatistiksel geçerlilik: FAIL grubu küçüktür ($n=17$); duyarlılık analizi (Bölüm IV-B) yalnızca orta-büyük etkilerin ($|r| \geq 0.46$) saptanabileceğini gösterir. Üç çıkarımsal test (Mann-Whitney, Fisher, McNemar) keşif amaçlıdır; Bonferroni düzeltmesinde ($\alpha=0.05/3 \approx 0.0167$) yalnızca McNemar ($p=0.013$) anlamlı kalır, yorum değişmez. Dış geçerlilik: veri seti 57 program/233 girdiyle sınırlıdır, endüstriyel kod tabanlarına genellenemez; GPT-4o/DeepSeek adaptörleri API erişimi olmadan çalıştırılamamış, çoklu-model bulgusu iki modelle sınırlı kalmıştır. Yapı geçerliliği: EA yalnızca gözlenen davranışsal eşdeğerliği ölçer; Seviye A oracle verdiğinden %100, yalnızca *görülen* girdilere karşı bir üst sınırdır — held-out doğrulama yapılmamıştır. Veri seti seçim yanlılığı: veri setinin 44/57'si bilinen boşlukları hedefler; dengeleyen 13 bağımsız örneğin 11'i geçmiştir. Kalıcı bir arşiv DOI'si henüz atanmamıştır; depo bağlantısı ve DOI kabul sonrası paylaşılacaktır (Bölüm III-F). Kategori C ve F'de “doğru davranış” referans platforma (Windows/LLP64) bağlıdır; bir kök neden değil bir artefaktır (Claude'un i64 seçimi Linux/LP64'te doğru olurdu, Bölüm IV-F). Model $\times$ kategori etkileşimi test edilmemiştir; Tablo VII'deki kırımlı betimseldir.

C. Öneriler

En iyi modelde bile (Gemini, %89.47) 57 programın 6'sında hata kalması, doğrulama ve iteratif düzeltmenin LLM destekli araç zincirlerinde zorunlu olması gerektiğini gösterir. Tablo VII'deki betimsel model $\times$ kategori kırımlı, tek bir “en iyi model” aramak yerine kod tabanının niteliğine göre model seçimi veya çoklu-model oylaması (ensemble) araştırılmasını önerir. Somut olarak: (i) derleme

başarısı yeterli değildir, diferansiyel test veya eşdeğeri bir yürütme tabanlı doğrulama zorunlu tutulmalıdır; (ii) dokuz kök-neden kategorisi (Tablo V) bir statik analiz/lint kuralı setine dönüştürülüp derleme hattına eklenebilir — özellikle derleyicinin yakalayamadığı unsigned aritmetik, char işaretliliği ve çıktı biçimlendirme için; (iii) uzun/karmaşık üretim kodunda bu kategoride daha dayanıklı bulunan model (Claude) tercih edilebilir, kısa izole fonksiyonlarda iki model de kabul edilebilir; ancak bu öneri yalnızca iki model ve 57 örneğe dayanır, daha geniş bir veri setiyle doğrulanmadan genel bir kural olarak benimsenmemelidir.

VI. SONUÇ

Bu çalışma, iki bağımsız LLM'in (Claude Sonnet 5, Google Gemini) ürettiği zero-shot C→Rust çevirilerinin yürütme doğruluğunu 57 programlık bir kıyaslama setinde (13'ü bağımsız gerçek dünya kodu) diferansiyel test ile ölçmüştür: Claude %70.18 (başarısızlıkların %94.1'i derleyiciden uyarı almadan, %70.6'sı çalışma zamanında da hiçbir sinyal vermeden geçmiştir), Gemini %89.47 (51/57) doğruluk göstermiştir. Kök neden analizi hataları dokuz tekrar eden C↔Rust semantik boşluğuna ayırmıştır; model $\times$ kategori karşılaştırması hiçbir modelin diğerinden tekdüze üstün olmadığını göstermiştir — Gemini kısa hedeflenmiş kategorilerde (5/5 vs. 1/5) daha dayanıklı, uzun gerçek üretim kodunda (0/3 vs. 2/3) daha kırılgandır; sessiz hata oranı da model-özgüdür (Claude: derlemeye yakalanmayan %94.1, çalışma zamanında da sinyalsiz %70.6; Gemini: her iki katmanda da %33.3). Hata geri bildirimi Claude'da doğruluğu %100'e taşımış, ama kısıtlı geri bildirimde bu %71.9-86.0'a düşmüştür. Bağımsız bir ikinci Claude denemesi ($k=2$) aynı 57 örnekte EA'yı %96.49'a taşımış, $k=1$ 'in tüm hatalarını düzeltirken 2 yeni ve farklı hata üretmiştir — tek-örneklemli EA ölçümlerinin (bu çalışmanın dahil) yüksek varyansa sahip olabileceğini doğrudan göstermiştir. Katman-eleştirilmiş bir karşılaştırma da (Claude Haiku, %63.16) Gemini Flash'ın gerisinde kalmış, ayrıca Rust'a özgü ödünç-denetleyici hatalarının küçük modellerde ayrı bir başarısızlık kaynağı olduğunu ortaya koymuştur. Gelecek çalışmalar GPT-4o/DeepSeek dahil daha geniş bir model havuzunu, $k \geq 3$ pass@k protokollerini ve model-seçim/ensemble stratejilerini hedefleyebilir.

KAYNAKÇA

- [1] Cybersecurity and Infrastructure Security Agency (CISA), National Security Agency (NSA), Federal Bureau of Investigation (FBI) et al., “The Case for Memory Safe Roadmaps: Why both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously,” joint cybersecurity guidance, Dec. 2023. [Online]. Available: <https://cisa.gov/case-memory-safe-roadmaps>. [Accessed: Jul. 25, 2026].
- [2] S. Gandhi, M. Patwardhan, J. Khatri, L. Vig, and R. K. Medicherla, “Translation of Low-Resource COBOL to Logically Correct and Readable Java leveraging High-Resource Java Refinement,” in Proc. 1st Int. Workshop on Large Language Models for Code (LLM4Code '24), ICSE 2024, Lisbon, Portugal, pp. 46-53, doi: 10.1145/3643795.3648388.
- [3] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, “Towards Translating Real-World Code with LLMs: A Study of Translating to Rust,” arXiv:2405.11514, 2024.
- [4] X. Yin, C. Ni, T. N. Nguyen, S. Wang, and X. Yang, “Rectifier: Code Translation with Corrector via LLMs,” arXiv:2407.07472, 2024.
- [5] Immunant, Inc., “C2Rust: migrate C code to Rust,” open-source project. [Online]. Available: <https://github.com/immunant/c2rust>. [Accessed: Jul. 25, 2026].
- [6] B. Rozière, M.-A. Lachaux, L. Chanussot, and G. Lample, “Unsupervised Translation of Programming Languages,” in Advances in

- Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 20601-20611.
- [7] Rosetta Code, rosettacode.org — Luhn test, Soundex, Levenshtein distance, 0-1 Knapsack, Towers of Hanoi, LCS, CRC-32 (C solutions). [Online]. Available: <https://github.com/acmeism/RosettaCodeData>. [Accessed: Jul. 25, 2026].
- [8] The Regents of the Univ. of California / OpenBSD and FreeBSD Projects, `getopt.c` (1987), `strtol.c` (1990), `heapsort.c` (1991). [Online]. Available: <https://github.com/openbsd/src>, <https://github.com/freebsd/freebsd-src>. [Accessed: Jul. 25, 2026].
- [9] musl libc Project (R. Felker et al.), `qsort.c` (smoothsort, V. Ochs, 2011). [Online]. Available: <https://git.musl-libc.org/cgit/musl/tree/src/stdlib/qsort.c>. [Accessed: Jul. 25, 2026].
- [10] S. Sanfilippo et al., `sds.c/sds.h` (SDS v2.0), Redis 7.2.4. [Online]. Available: <https://github.com/redis/redis> (tag 7.2.4). [Accessed: Jul. 25, 2026].
- [11] D. Gamble et al., `cJSON.c` (`parse_number`, `print_number`). [Online]. Available: <https://github.com/DaveGamble/cJSON>. [Accessed: Jul. 25, 2026].
- [12] Google, “Gemini API — Models,” Google AI for Developers, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/models>. [Accessed: Jul. 25, 2026].
- [13] J. M. Hoenig and D. M. Heisey, “The Abuse of Power: The Pervasive Fallacy of Power Calculations for Data Analysis,” *The American Statistician*, vol. 55, no. 1, pp. 19-24, 2001.